

Setup a Raspberry Pi for headless use with USB serial console

11/16/2017 - 18:14

Changed: 02/26/2018 - 16:26

This tutorial will describe how to configure a serial console over USB on Raspberry Pi Zero, Zero W, A and A+ models (Also Compute Module 3). Note that these are the only models where it is possible to do so as they don't have a USB hub and therefore OTG mode can be used. For serial console on other models you need to use a USB TTL or RS-232 adapter unfortunately.

The Raspberry Pi *Zero*, *Zero W*, *A* and Raspberry Pi *A+* are great cheap little devices, but connecting to them without any screen and keyboard can be a bit of a pain. You can always use a TTL RS-232 adapter but using just one simple USB cable is much more easier and convenient. And as the Zero & Zero W comes without any pin headers you have the additional hassle of needing to solder one.

Thankfully it is possible to configure them to use [USB gadget mode](#) to provide a *serial console* over the USB connector (or even a [USB ethernet connection](#) if needed).

Unfortunately it won't work as system console as the gadget driver is loaded as a module, but we can use it for serial console.

This tutorial will show how to do that on plain SD card under Linux. You can also modify the SD card image directly in case you need to make the modification for multiple SD cards.

Prepare a Raspbian SD card

First, [prepare a SD card of raspbian](#) as your normally would, I will not go into the details here as there are [plenty of sources](#) for that. You can use **full** or **lite** version depending on your needs, both will work.

Mount the SD card partitions

When done preparing the SD card, remove the card and re-insert so the system will re-read the new partition scheme of the SD card.

Then mount the filesystems, this depend a bit on your environment, on Ubuntu and derivatives like Linux Mint the root and boot partitions will be mounted automatically.

If they are not, check in the dmesg the devices nodes that the SD card shows up as.

For example if the card is available as a **mmc** device nodes the partitions will show up as something looking like this:

```
mmcblk0: mmc0:59b4 USDU1 15.0 GiB
  mmcblk0: p1 p2

brw-rw---- 1 root disk 179, 0 Nov 16 11:17 /dev/mmcblk0
brw-rw---- 1 root disk 179, 1 Nov 16 11:17 /dev/mmcblk0p1
brw-rw---- 1 root disk 179, 2 Nov 16 11:17 /dev/mmcblk0p2
```

p1 is the /boot partition. We will use **BOOT** to point to the mount point of this partition below.

p2 is the / partition. We will use **ROOTFS** to point to the mount point of this partition below.

If you are using a SD to USB adapter they will most likely show up as /dev/sdb1 and /dev/sdb2 (or sdc,

sde, etc. Depending on amount of drives in your system)

Mount these as you normally would in case your system didn't auto mount them. Location is up to you.

Optional, modify Raspbian SD image in place

You can also modify the image directly, before writing it to a SD card. For example if you need this feature on multiple Pies. In this case, we need to configure a [loopback device](#) and mount the image partitions.

First, configure a loopback device with losetup for the Raspbian image you are using:

```
sudo losetup -f --show -P 2017-09-07-raspbian-stretch-lite.img
```

Note the loopback device node the losetup command will print out, it is usually something close to `/dev/loop0`. Next, mount the SD image loopback device partitions, `/dev/loop0p1` (**BOOT**) and `/dev/loop0p2` (**ROOTFS**).

Now make the below modifications inside these mountpoints, unmount, detach the image from the loopback device (`sudo losetup -d /dev/loop0`) and then write the image to your SD card.

Configure Raspbian image

Next we need to adjust the configuration of the image, enable USB OTG driver, enable USB serial gadget driver and last enable a console on the USB serial device so we can login.

USB Driver setting

We need to use the **dwc2** USB controller driver, luckily on the Pi Zero dwc2 is the default driver so we don't need to change anything for that. Should be default according to documentation, but it didn't work for me, so:

Enable **dwc2** driver in **BOOT**/config.txt, add the line at the end. At the same time you can enable/disable anything extra you might need like SPI or I2C.

```
dtoverlay=dwc2
```

Modify default cmdline.txt to enable serial gadget driver

We need to enable loading of the serial USB gadget driver.

In **BOOT**/cmdline.txt

Add "**modules-load=dwc2,g_serial**" to the kernel command line file, cmdline.txt in /boot and remove the "init=/usr/lib/raspi-config/init_resize.sh" part as otherwise the first boot will resize the root partition and the USB drivers won't engage until the next boot and it will first look like the configuration failed. Note the root=PARTUUID=xxxxxxxx-yy part, don't modify that.

The resulting command line in *cmdline.txt* should look something close to this :

```
dwc_otg.lpm_enable=0 console=serial0,115200 console=tty1 root=PARTUUID=72d5f72a-02
rootfstype=ext4 elevator=deadline fsck.repair=yes rootwait modules-load=dwc2,g_serial
```

Optionally, if you need networking over USB, use [g_cdc](#) instead of [g_serial](#)

Enable serial console login on USB gadget serial node

If we where on a live running system it would be a matter of running the [systemd](#) command:

```
sudo systemctl enable getty@ttyGS0.service
```

But as we are modifying the filesystem on another computer we need to manually enable [getty](#) on ttyGS0

Go to *(ROOT)/etc/systemd/system/getty.target.wants* and create a symlink

```
ln -s /lib/systemd/system/getty@.service getty@ttyGS0.service
```

Boot with serial console

We are now done, umount the partitions and insert the SD card into your Raspberry PI and connect a USB cable to the USB port (**not** the one marked POWER!). Your Pi should now boot and a USB serial device should pop up on the host device after a short while, start your terminal software (minicom, picocom) and point it to the new device node, probably /dev/ttyACM0. Check dmesg.

Press ENTER and you should be presented with a login prompt.

Problems ?

- Raspbian is quite slow to boot on the Pi Zero, it might take a while before the serial device appears as it will be loaded quite late in the boot stage.
- Check that you did everything right
- If possible, connect a monitor and see what happens.
- If a message, "udc-core: couldn't find an available UDC - added [g_serial] to list of pending drivers." is printed, then you didn't enable the **dwc2** driver properly.

[Printer-friendly version](#)

Book traversal links for Setup a Raspberry Pi for headless use with USB serial console

- [◀ Raspberry Pi Tutorials](#)
- [Up](#)
- [Build 64-bit kernel for Raspberry Pi, using native tools ▶](#)

Book navigation

- [Setup a Raspberry Pi for headless use with USB serial console](#)
- [Build 64-bit kernel for Raspberry Pi, using native tools](#)
- [Booting 64-bit kernel on Raspberry Pi 3](#)
- [Setup PHP 8 with Apache on a Raspberry Pi](#)
- [Building Qt 5.12 LTS for Raspberry Pi on Raspberry Pi OS](#)
- [Building Qt 5.15 LTS for Raspberry Pi on Raspberry Pi OS](#)

[Tutorials](#) | [Projects](#) |
[Privacy policy](#)

- [Follow site on Mastodon](#)
- [Follow author on Mastodon](#)

www.tal.org is the **home of a freelance coder** and tinker that creates various applications for various platforms and devices.

As an Amazon Associate I earn from qualifying purchases.

- **Follow on YouTube**
- **Follow on Instagram**

Log in